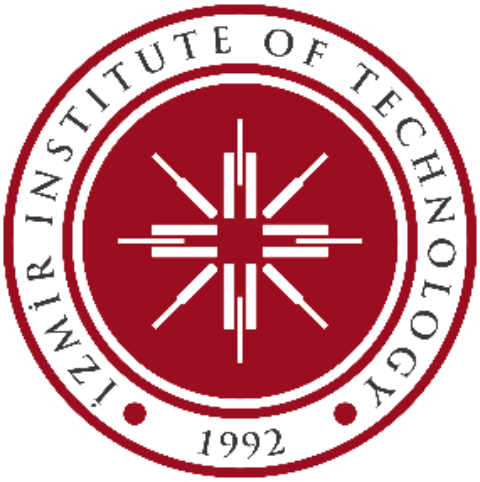

EE_331 FINAL LAB REPORT

Student Name Surname: Elif Sude ÖZTÜRK

Student no: 310206045

Submission Date: 11.01.2026

1) INTRODUCTION:

In this experiment, an audio signal consisting of eight successive musical notes was recorded and analyzed using MATLAB. The main objectives of the study are to analyze the signal in both time and frequency domains, determine the fundamental frequencies of each note, compare the measured frequencies with theoretical values, and design a digital filter to isolate a selected note. Additionally, a graphical user interface (GUI) was developed to visualize the signals and interactively apply filtering operations.

2) DATA COLLECTION:

The sound was recorded using the “voice recorder” application on a Samsung Galaxy S25 device, played between the C4 and C5 notes, approximately 10 cm away from the piano's E4 note, from above. The note range was held for approximately 1 second. The recorded audio signal was saved as a WAV file and loaded into MATLAB using the **audioread** function.

3) METHOD:

The complete audio signal was first examined in the time domain to observe the overall structure of the recording. The waveform clearly shows the presence of eight successive notes separated by transitions. In addition to the full signal, a short time segment was selected and plotted to better visualize waveform details such as amplitude variations and note onsets.

To analyze each note separately, the recorded signal was segmented into eight individual parts using manually defined time boundaries corresponding to the start times of each note. The duration of each note was calculated from these boundaries.

Since the durations of the notes were not exactly equal, the minimum note duration was selected, and an equal number of samples was extracted from each note segment. This ensured consistent frequency resolution for all notes during spectral analysis. Each segmented note was plotted separately in the time domain.

The frequency content of the recorded signal was analyzed using the Fast Fourier Transform (FFT). First, the FFT of the entire signal was computed and shifted using fftshift so that the zero-frequency component was centered. The magnitude spectrum was plotted to observe the overall frequency distribution of the recorded notes. For a more detailed analysis, the FFT of each segmented note was computed individually. The magnitude spectrum of each note was plotted, and the dominant spectral peak was identified. This peak was taken as the fundamental frequency of the corresponding musical note.

The measured fundamental frequencies obtained from the FFT analysis were compared with the theoretical frequencies of the musical notes. The percentage error for each note was calculated by normalizing the absolute difference between the measured and theoretical frequencies with the theoretical value. The measured frequencies, theoretical

frequencies, and percentage errors were visualized in the “command window” by creating a frequency comparison table to evaluate the accuracy of the analysis.

The Butterworth bandpass filter is designed to isolate the selected “target” musical note from the original signal. The filter's center frequency is selected based on the measured fundamental frequency of the target note. A fixed bandwidth of ±15 Hz around the center frequency is used to define the passband.

The filter order was selected as 10 to achieve sufficient frequency selectivity. To increase numerical stability, the filter was implemented using second-order sections (SOS). The reason for this is that using the Butterworth filter with the direct filter or filter function causes problems in band-pass filters. Band-pass systems require an order of 4 or higher, which is supported by Matlab with its original documentation[1]. For this purpose, the SOS structure is used. This allows us to obtain cleaner and smoother results. The filter designed in this way was applied to the entire audio signal.

To observe the filtering effect, the original and filtered signals were compared in the time domain using graps. The filtered signal graph shows that while the target note is emphasized, other notes are significantly attenuated.

For the frequency domain comparison, the mean value of each signal was removed, and a Hann window was applied before calculating the FFT. The magnitude spectra of the original and filtered signals were plotted using only the positive frequency components, allowing for a clear comparison of the filter's effect on the spectrum. The positive part was used because the FFT form of the signal is symmetric at 0 Hz in the frequency domain. Therefore, we used the positive part of the FFT form.

4) GUI METHOD:

To make the filtering process more interactive, a graphical user interface (GUI) was developed using MATLAB App Designer. The GUI allows the user to load a WAV file and automatically display the original signal in both the time and frequency domains. The sampling frequency, total duration, and number of samples are displayed on the interface.

The GUI provides controls for selecting the filter type (bandpass, lowpass, or highpass), filter order, and target musical note. After the selected filter is applied, the filtered signal is displayed in both the time and frequency domains. The GUI also includes a playback function that allows the user to listen to both the original and filtered signals separately.

During the GUI development process, MATLAB's official documentation, online resources, and artificial intelligence sources were used to understand the behavior and properties of the **uitable** component and integrate it with the filter.[2][3]

5) RESULTS:

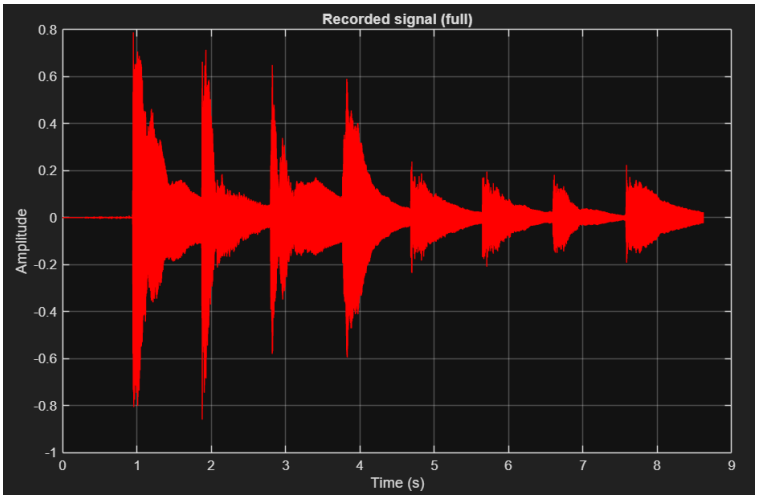

Figure 1: Recorded Signal at Time-Domain

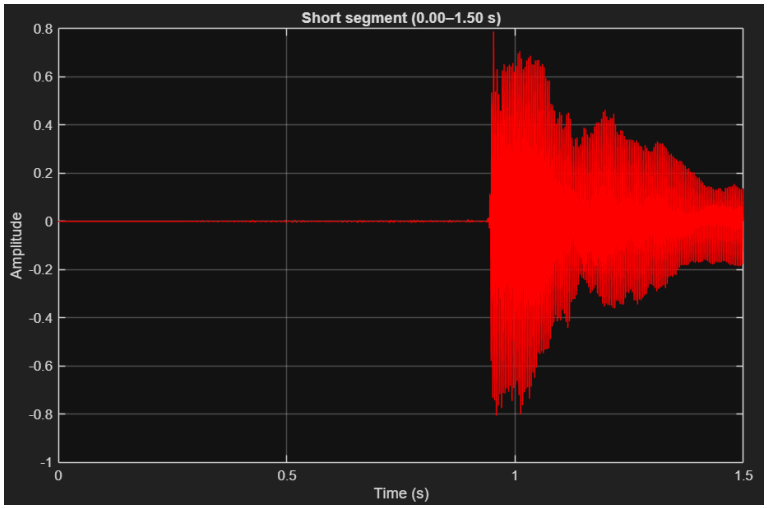

Figure 2: Short Segment

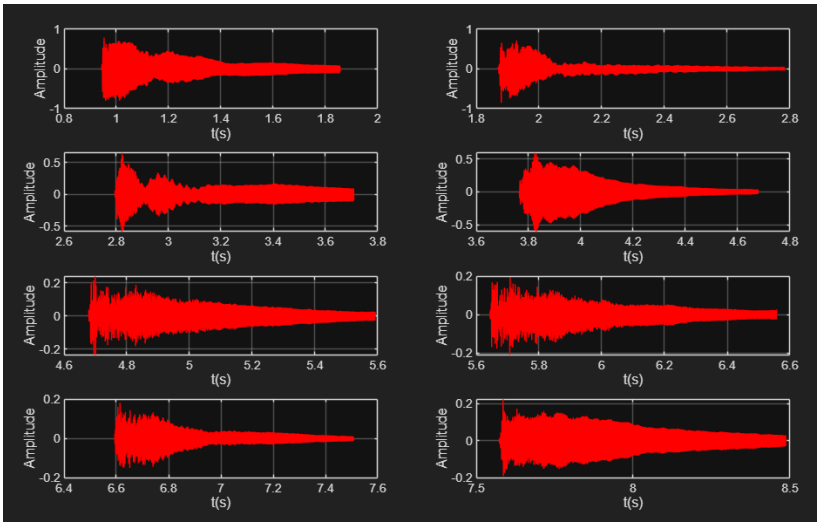

Figure 3: Each Note Signal at Time-Domain

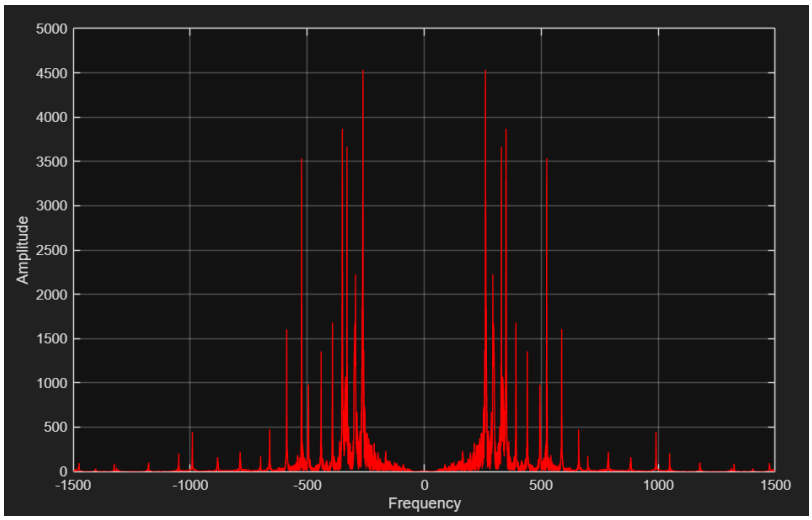

Figure 4: FFT of Original Signal

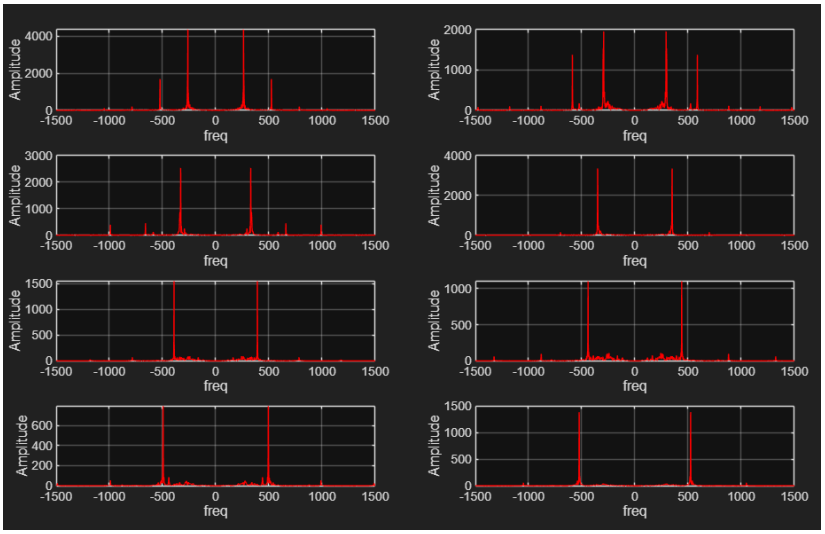

Figure 5: FFT of Each Note Signal

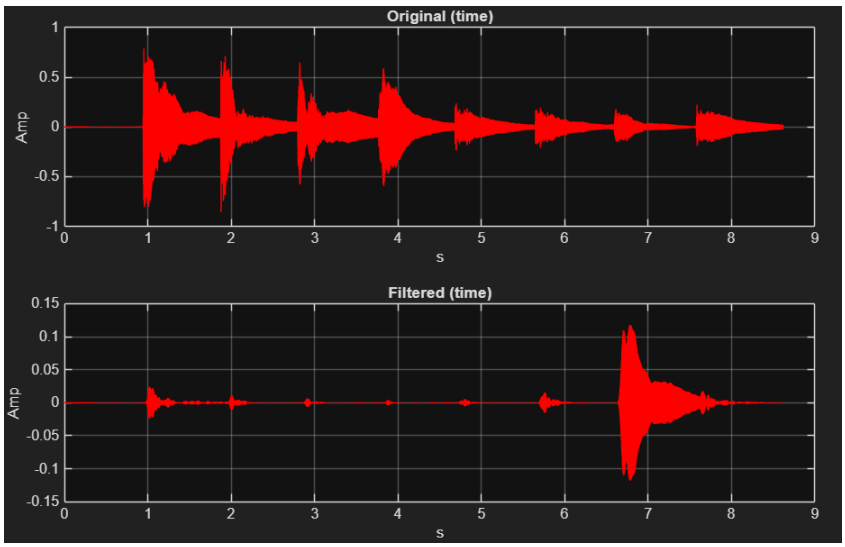

Figure 6: Time Comparison(B4)

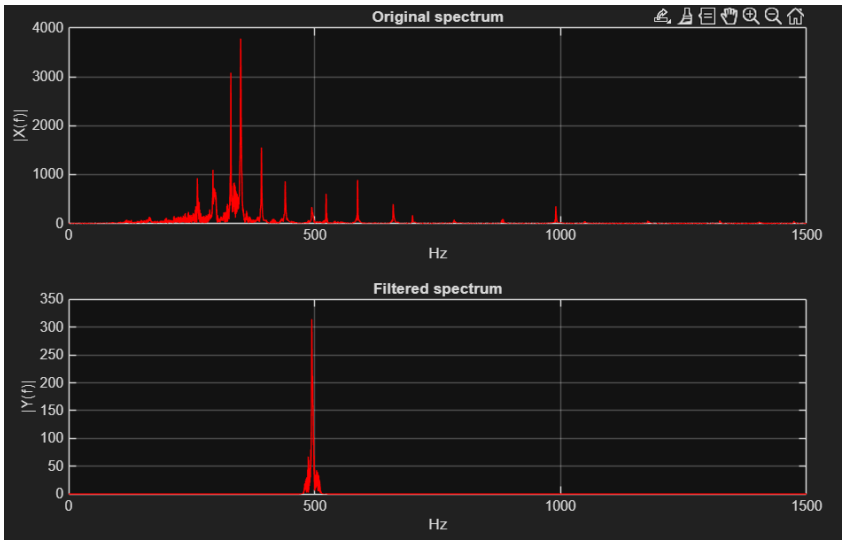

Figure 7: Spectrum Comparison

==== Frequency Table ====

Index	Note	Measured_Hz	Theoretical_Hz	Error_percent
1	"C4"	261.55	261.63	0.031502
2	"D4"	293.28	293.66	0.12822
3	"E4"	329.4	329.63	0.070763
4	"F4"	350.19	349.23	0.27467
5	"G4"	391.77	392	0.057602
6	"A4"	439.93	440	0.016995
7	"B4"	494.64	493.88	0.15435
8	"C5"	524.19	523.25	0.17955

Figure 8: Frequency Table

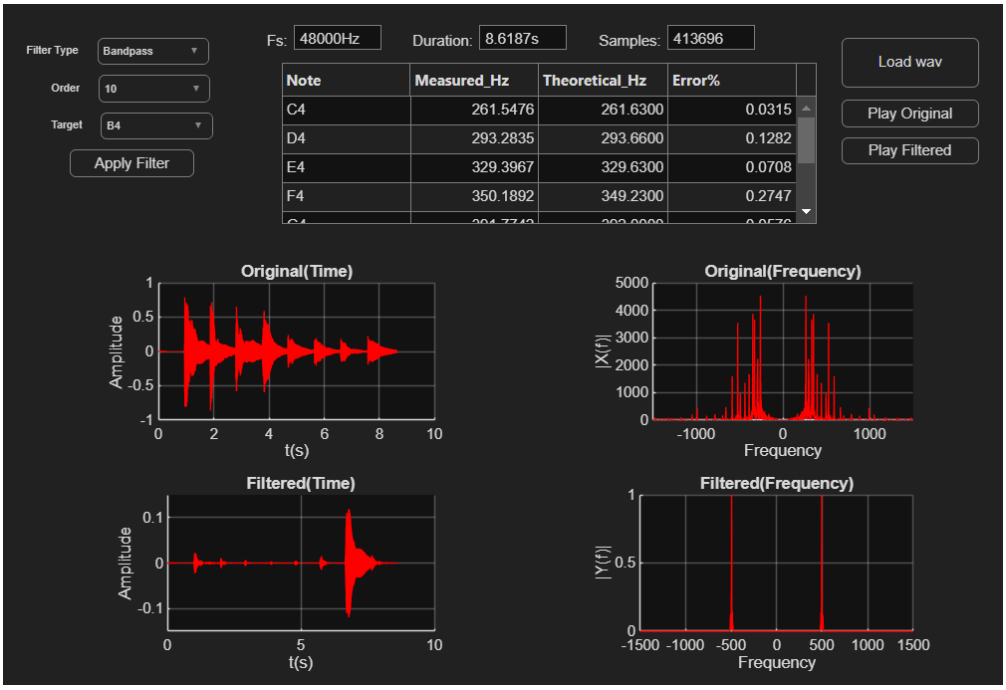

Figure 9: GUI

6) DISCUSSION AND CONCLUSION:

Small differences were observed between the measured and theoretical frequencies. These differences can be attributed to FFT resolution limitations, background noise during recording, the instrument's tuning not being ideal, minor errors in manual segmentation, and the presence of harmonic components.

The Butterworth bandpass filter effectively isolated the selected note, and the use of second-order sections ensured stable operation of the filter. The GUI application provided an interactive way to visualize and apply signal processing techniques.

Overall, this assignment provided real-life experience in analyzing real audio signals, measuring fundamental frequencies and comparing them with ideal case frequencies, designing digital filters, and developing a user-friendly GUI for signal processing applications.

7) REFERENCES:

[1] MathWorks, *butter — Butterworth digital and analog filter design*, MATLAB Signal Processing Toolbox Documentation.
Available: https://www.mathworks.com/help/signal/ref/butter.html

[2] MathWorks, *uitable — Create UI table*, MATLAB Documentation.
Available: https://www.mathworks.com/help/matlab/ref/uitable.html

[3] GeeksforGeeks, *Control Table UI Component Appearance and Behavior in MATLAB*.
Available: https://www.geeksforgeeks.org/software-engineering/control-table-ui-component-appearance-and-behavior-in-matlab/